

Bangladesh University of Engineering and Technology (BUET)

Department of Computer Science and Engineering

Design Report

Model Extraction: Stealing a Black-Box Classifier

(Prediction-API Model Stealing)

Group ID	9 (Section B2)
Group members	Pritu Dhar Dhip (2105109) Arian Mahtab Chowdhury (2105106)
Course	CSE406: Cyber Security Sessional
Submission Date	11th September, 2026

1. Overview of the Project Idea

1.1 Summary

This project designs a **model extraction** (model-stealing) attack against a machine-learning classifier that is exposed only through a network **prediction API**. A model owner trains a secret classifier f and lets clients query it — input in, prediction out — while keeping the weights and training data confidential. The attacker has **black-box access only**: it sends inputs, records the returned predictions, and uses those input–prediction pairs to train a substitute (“knockoff”) model $\hat{f}$ that reproduces the victim’s functionality, without ever seeing the weights or the training set.

1.2 Objective

Show that an adversary with nothing but query access to a paid prediction API can reconstruct a functionally equivalent copy of the model at a fraction of the original training cost — and identify the layered defences that raise that cost. Everything graded (victim server, custom wire protocol, query engine, extraction/training loop, and defence) is implemented by us; PyTorch and scikit-learn are used only as the environment. No ready-made attack library (foolbox, ART, TextAttack) is used.

1.3 Scope and threat model

- **Actors.** Two hosts on one segment — the *Victim* (serves the secret classifier over a TCP prediction API) and the *Attacker* (an ordinary client).
- **Attacker capability.** Black-box query access only: it may submit inputs and read back the prediction vector. It knows neither the weights, the gradients, nor the training data.
- **Assumption.** The API returns a class-probability (confidence) vector, the common MLaaS pattern; a label-only variant is handled as a harder case.

1.4 Ethical and legal scope

This design targets only a model **owned by the authors**, served on a private, isolated host. Extracting a third party’s paid model without authorisation violates its terms of service and may be unlawful; the attack is documented here to explain the mechanism and to justify the defences in Section 6.

2. Network Topology, Components and Timing Diagrams

2.1 Topology

The attacker and victim communicate over TCP (Figure 1). The attacker repeatedly sends a **REQUEST** frame carrying one input; the victim runs inference, passes the result through a defence layer, and returns a **RESPONSE** frame carrying the (possibly degraded) prediction. All observed pairs accumulate in a local *transfer set* that later trains the knockoff offline.

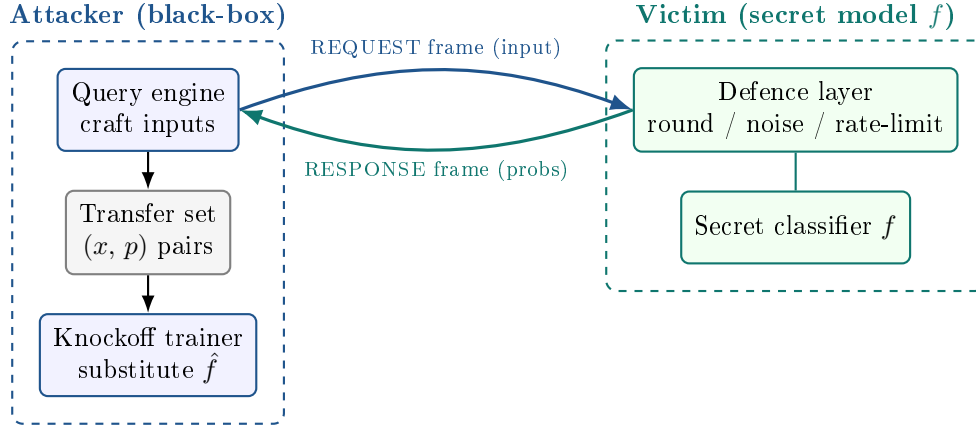


Figure 1: Attacker–victim topology. The attacker holds only a socket; every input is answered by a prediction that (optionally) passes through the victim’s defence layer.

2.2 Components

Host	Role	Responsibility
Victim	Server	Loads the secret CNN, listens on TCP, decodes REQUEST frames, runs inference, applies the defence, returns RESPONSE frames.
Attacker	Client	Runs the self-built query engine (crafts and streams inputs), the frame parser (records prediction vectors), and the knockoff trainer.

The victim answers every well-formed query, so the attacker cannot be blocked by a single bad request; the only lever the victim holds is *how much information each answer leaks* — exactly what the defences in Section 6 restrict.

2.3 Timing — query campaign vs. extraction

Figure 2 shows the two phases. In the *online* phase the attacker repeats a request–response round q times, building the transfer set. In the *offline* phase — with no network at all — it trains $\hat{f}$ on those pairs and evaluates accuracy, fidelity (label agreement with the victim), and adversarial-example transfer.

3. Packet and Frame Structure

TCP provides a reliable byte stream but no message boundaries, so we define our own length-prefixed application-layer framing. Every message begins with a fixed 12-byte header; all multi-byte fields are big-endian (network order). The header **struct** format is `>HBBII`.

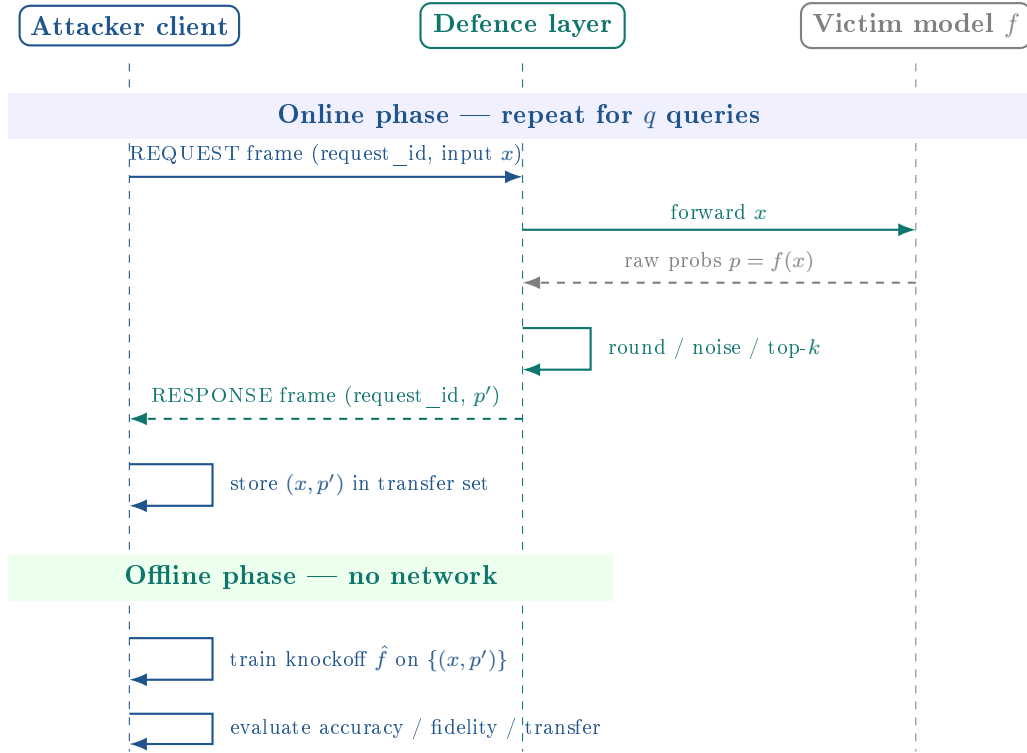


Figure 2: Attack timeline: query $\rightarrow$ record $\rightarrow$ (offline) train $\rightarrow$ evaluate. The base attack is *non-adaptive* (inputs chosen in advance), which helps it bypass query-pattern detectors.

3.1 Common header (12 bytes)

Off	Field	Size	Type	Meaning
0	MAGIC	2 B	uint16	0x4D58 (“MX”): version/sanity guard; drop if mismatch.
2	VERSION	1 B	uint8	protocol version = 1.
3	MSG_TYPE	1 B	uint8	1 = REQUEST, 2 = RESPONSE, 3 = ERROR.
4	REQUEST_ID	4 B	uint32	matches a response to its request (pipelining).
8	PAYLOAD_LEN	4 B	uint32	number of payload bytes that follow.

On receive: read exactly 12 header bytes, parse `PAYLOAD_LEN`, then loop `recv()` until that many payload bytes have arrived — this reassembles messages across TCP segmentation/coalescing.

3.2 REQUEST and RESPONSE payloads

REQUEST payload (attacker → victim)

H	2 B	uint16	image height (e.g. 28).
W	2 B	uint16	image width (e.g. 28).
C	1 B	uint8	channels (1 for grayscale).
PIXELS	$H \cdot W \cdot C \cdot 4$ B	float32[]	row-major normalized pixels in $[0, 1]$.

RESPONSE payload (victim → attacker)

K	2 B	uint16	number of classes (e.g. 10).
FLAGS	1 B	uint8	bit0 = probs present, bit1 = top- k truncated, bit2 = noised.
PROBS	$K \cdot 4$ B	float32[]	class probabilities p' after the defence layer.

3.3 Field layout of a REQUEST frame

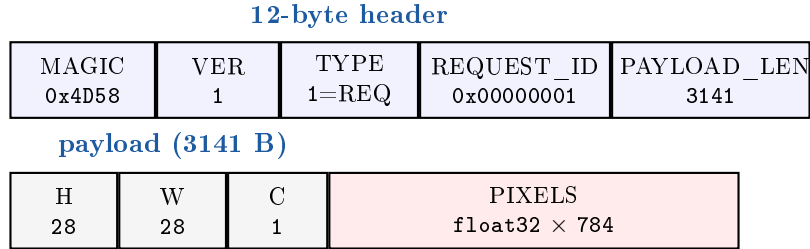


Figure 3: A REQUEST frame for a 28×28 grayscale input. Payload length = $5(H, W, C) + 784 \times 4 = 3141$ bytes. The self-describing header lets the receiver reassemble the exact byte count off the stream.

3.4 The information the response leaks

The prediction vector is the whole game. A full PROBS vector such as $[0.01, 0.92, 0.02, \dots]$ is a rich soft label that trains the knockoff fast; a label-only response (`argmax`) still leaks the decision but forces many more queries. The FLAGS byte records which defence (rounding, top- k , noise) was applied — each one, added in Section 6, removes information the attack depends on.

4. Implementation Plan

4.1 Laboratory build

1. Train the secret classifier on a public dataset (MNIST victim; Fashion-MNIST / uniform noise as an out-of-distribution query pool). Fix `SEED = 1337` for NumPy, PyTorch and Python `random`; save `victim.pt` and record its test accuracy.
2. Serve `victim.pt` over TCP with the self-built framing; verify one round-trip with a hard-coded input.
3. Point the attacker at the socket and confirm it receives well-formed RESPONSE frames.

4.2 Components

Component	Owner	Purpose
A. Victim server (<code>victim_server.py</code>)	Pritu	Loads the secret CNN, decodes frames, runs inference, applies the defence, returns predictions.
B. Protocol (<code>protocol.py</code>)	shared	Own binary framing: pack/unpack header + payload, length-prefix re-assembly.
C. Query engine + parser (<code>attack_client.py</code>)	Arian	Streams inputs, parses responses, writes the transfer set to disk.
D. Knockoff trainer (<code>knockoff_train.py</code>)	Arian	Trains $\hat{f}$ on the transfer set via soft-label distillation.
E. Equation-solver (<code>equation_solve.py</code>)	Pritu	Optional: recovers a linear victim's weights via least squares.
F. Defence (<code>defence.py</code>)	Pritu	Rounding, noise, top- k truncation, per-client query monitor.

4.3 Key algorithms

The query engine records input–prediction pairs; the trainer distils them into $\hat{f}$:

```
# C. Query engine (black-box: no weights, no gradients)
transfer = []
for x in query_inputs[:q]:    # non-adaptive: fixed set
    p = send_query(sock, x)    # our protocol round-trip
    transfer.append((x, p))

# D. Knockoff training (distillation on the victim's soft labels)
f_hat = SmallCNN()           # may differ from victim architecture
for (x, p) in batches(transfer):
    loss = soft_ce(f_hat(x), p, T) # temperature T
    loss.backward(); optim.step(); optim.zero_grad()
```

For a linear/softmax victim, $\text{logit}_i(x) = w_i \cdot x + b_i$; querying $d+1$ independent inputs and reading back the confidences gives a solvable linear system per class, recovering w_i, b_i by least squares (Attack B, functionally equivalent).

4.4 Execution

1. Start the victim server; confirm a single query returns a valid probability vector.
2. Run the query engine for a budget sweep $q \in \{1k, 5k, 10k, 30k\}$, saving `transfer_set.npz`.
3. Train $\hat{f}$; evaluate accuracy, fidelity and adversarial-example transfer.
4. Repeat every run with the defence layer on vs. off to quantify its effect.

5. Expected Outcome of a Successful Attack

The attacker obtains a substitute model that mirrors the victim's behaviour at a fraction of the original training cost:

Attack	Victim	Expected result
A (learning)	CNN, in-distribution queries	$\hat{f}$ reaches $\approx 0.90\text{--}0.98\times$ victim accuracy at 10k–30k queries.
A (learning)	CNN, out-of-distribution queries	still usable ($\approx 0.8\times+$): a diverse query pool works even off-distribution.
A (learning)	CNN	fidelity high but $< 100\%$ — bounded to $\approx 93\text{--}94\%$ by learning non-determinism.
A (learning)	CNN	adversarial examples crafted on $\hat{f}$ fool the victim well above chance.
B (equation)	softmax / logistic regression	$\approx 100\%$ agreement (functionally equivalent), a few hundred queries.

Why it works. A prediction API returns, per query, far more than a bystander could guess: a confidence vector is a supervised training signal handed to the attacker for free. The server cannot tell a learner’s queries from a paying customer’s, so a patient attacker reconstructs the model one answer at a time. This single fact is what the defences below target.

6. Defense Idea

Defences live in the victim’s `defence.py` and are evaluated with and without each measure to show the accuracy/robustness trade-off.

6.1 Output minimisation — the primary control

Return less per query: round probabilities to r decimals, expose only the top- k classes, or return the `argmax` label alone. Rounding to 2–3 decimals or dropping confidences sharply raises the query cost of learning- and equation-based attacks. This is the primary control, but only *friction*: a determined label-only attacker still extracts a usable model, just slowly.

6.2 Prediction perturbation

Add small calibrated noise to the returned vector so the `argmax` is preserved for honest users but the soft-label signal the distillation attack relies on is corrupted, lowering the knockoff’s fidelity.

6.3 Query-pattern monitoring and rate limiting

Track per-client query volume and whether inputs look unnaturally uniform or space-filling rather than natural data; flag or rate-limit deviating clients and enforce a per-client quota (ERROR code 4). Caveat: a non-adaptive attack using natural inputs can evade distribution-based detection — which is why output minimisation and monitoring are complementary, not alternatives.

6.4 Ownership verification (watermarking)

Embed a secret trigger set so a suspected stolen model can be *proven* to be a copy after the fact. This supports attribution rather than prevention, and is noted as future work.

Honest evaluation. No single defence stops extraction: output minimisation raises query cost, perturbation blunts fidelity, and monitoring catches naive high-volume attackers, but a patient, label-only attacker still succeeds slowly. The realistic goal — the same one the source literature adopts — is to make stealing *more expensive than paying*.

References

1. F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, T. Ristenpart. *Stealing Machine Learning Models via Prediction APIs*. USENIX Security 2016.
2. M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, N. Papernot. *High Accuracy and High Fidelity Extraction of Neural Networks*. USENIX Security 2020.
3. T. Orekondy, B. Schiele, M. Fritz. *Knockoff Nets: Stealing Functionality of Black-Box Models*. CVPR 2019.